FACULDADE DE TECNOLOGIA DO IPIRANGA “PASTOR ENÉAS TOGNINI”

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
(NOTURNO)

André Gondim Carlucci

ESTRUTURA DE DADOS

Atividade N1-2

Carlos Henrique Verissimo Pereira

São Paulo
2026

C (C17 + GNU extensions)
known limitations

```
14          novo->proximo = NULL;
15      }
16
17      return novo;
18  }
19
20  void erro_memoria() {
21
22      No *p = criar_no(10);
23
24  }
25
26  int main()
27  {
28      No *A = criar_no(10);
29      No *B = criar_no(20);
30      A->proximo = B;
31      free(B);
32
33      return 0;
34  }
```

Stack Heap

main

A	pointer to No ?
B	pointer to No ?

Note: ? refers to an uninitialized value

C/C++ details: none [default view]

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct No {
    int dado;
    struct No *proximo;
} No;

No* criar_no(int valor) {

    No *novo = (No*) malloc(sizeof(No));
    if (novo != NULL) {
        novo->dado = valor;
        novo->proximo = NULL;
    }

    return novo;
}

void erro_memoria() {
```

main

A	pointer to No ?
B	pointer to No ?

criar_no

valor	int ?
novo	pointer to No ?

Note: ? refers to an uninitialized value

C/C++ details: none [default view]

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct No {
    int dado;
    struct No *proximo;
} No;

No* criar_no(int valor) {

    No *novo = (No*) malloc(sizeof(No));
    if (novo != NULL) {
        novo->dado = valor;
        novo->proximo = NULL;
    }

    return novo;
}

void erro_memoria() {
```

main

A	pointer to No ?
B	pointer to No ?

criar_no

valor	int 10
novo	pointer to No ?

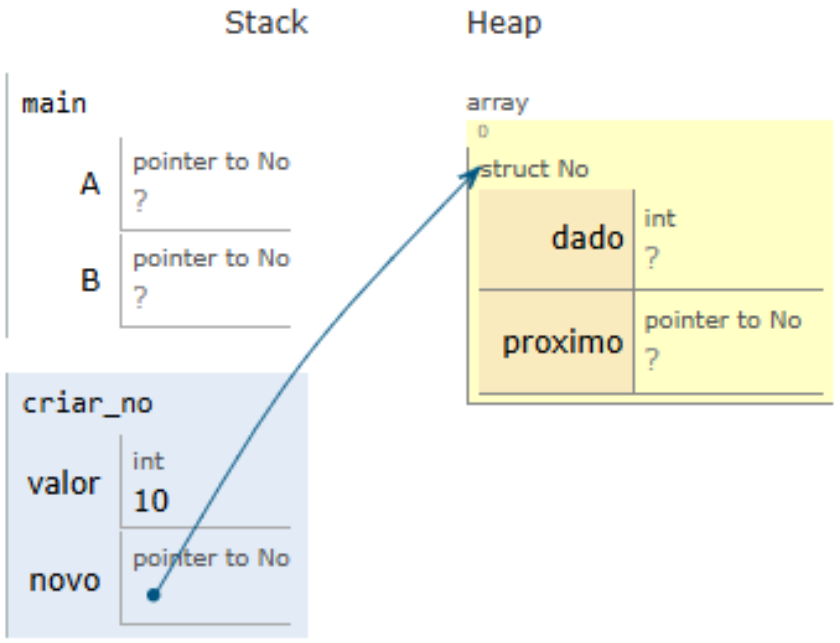
Note: ? refers to an uninitialized value

C/C++ details: none [default view]

C (C17 + GNU extensions)
known limitations

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct No {
    int dado;
    struct No *proximo;
} No;

No* criar_no(int valor) {

    No *novo = (No*) malloc(sizeof(No));
    if (novo != NULL) {
        novo->dado = valor;
        novo->proximo = NULL;
    }

    return novo;
}

void erro_memoria() {
```

Edit this code



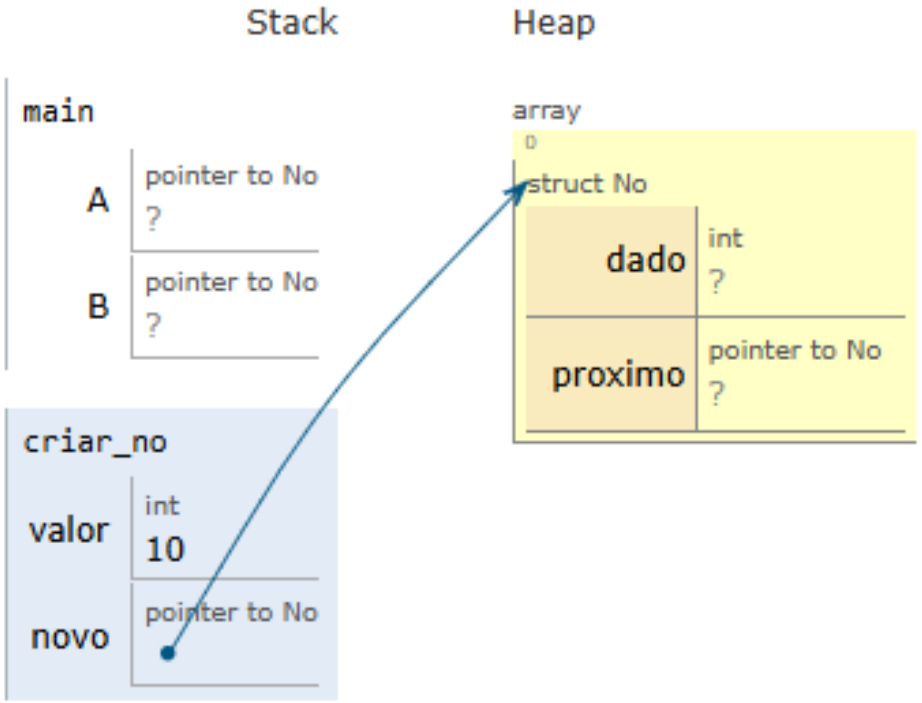
Note: ? refers to an uninitialized value

Note: Pointers to structs, unions, and C++ objects may be ambiguous since a struct/object's address is the same as its first member and a union's address is the same as all its members. Select 'show memory addresses' below to see more details:

C/C++ details: none [default view]

C (C17 + GNU extensions)
known limitations

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct No {
    int dado;
    struct No *proximo;
} No;

No* criar_no(int valor) {

    No *novo = (No*) malloc(sizeof(No));
    if (novo != NULL) {
        novo->dado = valor;
        novo->proximo = NULL;
    }

    return novo;
}

void erro_memoria() {
```



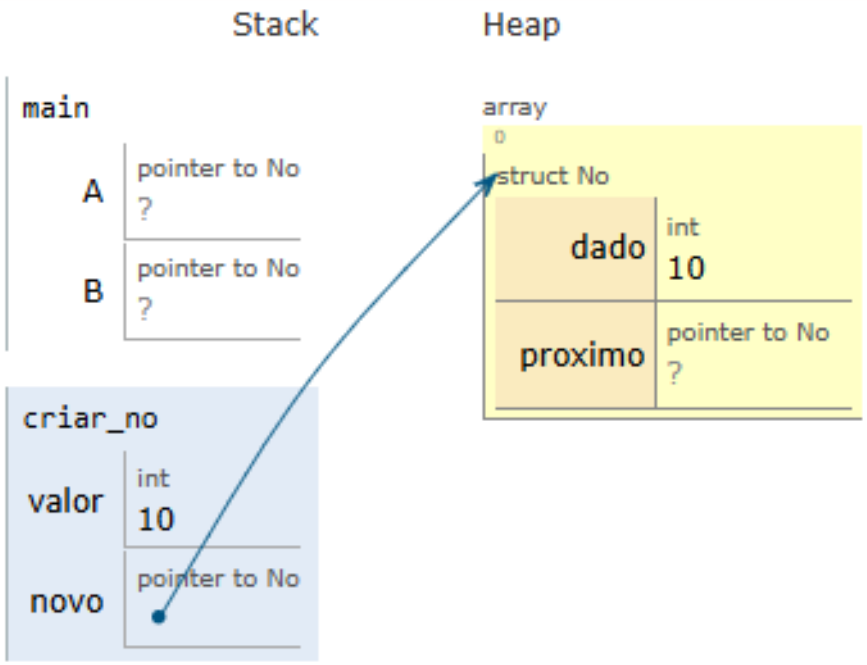
Note: ? refers to an uninitialized value

Note: Pointers to structs, unions, and C++ objects may be ambiguous since a struct/object's address is the same as its first member and a union's address is the same as all its members. Select 'show memory addresses' below to see more details:

C/C++ details: none [default view]

C (C17 + GNU extensions)
known limitations

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct No {
    int dado;
    struct No *proximo;
} No;

No* criar_no(int valor) {

    No *novo = (No*) malloc(sizeof(No));
    if (novo != NULL) {
        novo->dado = valor;
        novo->proximo = NULL;
    }

    return novo;
}

void erro_memoria() {
```



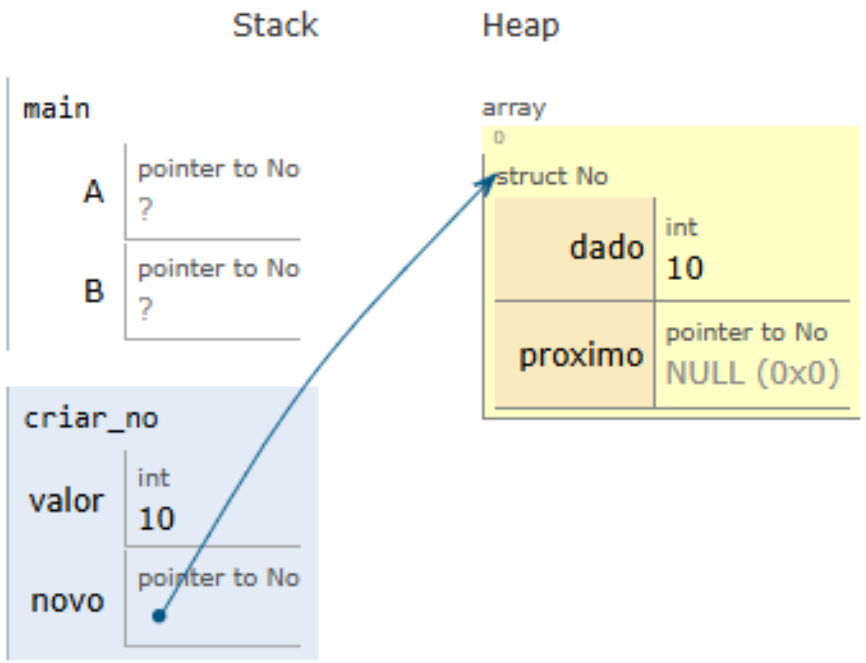
Note: ? refers to an uninitialized value

Note: Pointers to structs, unions, and C++ objects may be ambiguous since a struct/object's address is the same as its first member and a union's address is the same as all its members. Select 'show memory addresses' below to see more details:

C/C++ details: none [default view]

C (C17 + GNU extensions)
known limitations

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct No {
    int dado;
    struct No *proximo;
} No;

No* criar_no(int valor) {

    No *novo = (No*) malloc(sizeof(No));
    if (novo != NULL) {
        novo->dado = valor;
        novo->proximo = NULL;
    }

    return novo;
}

void erro_memoria() {
```



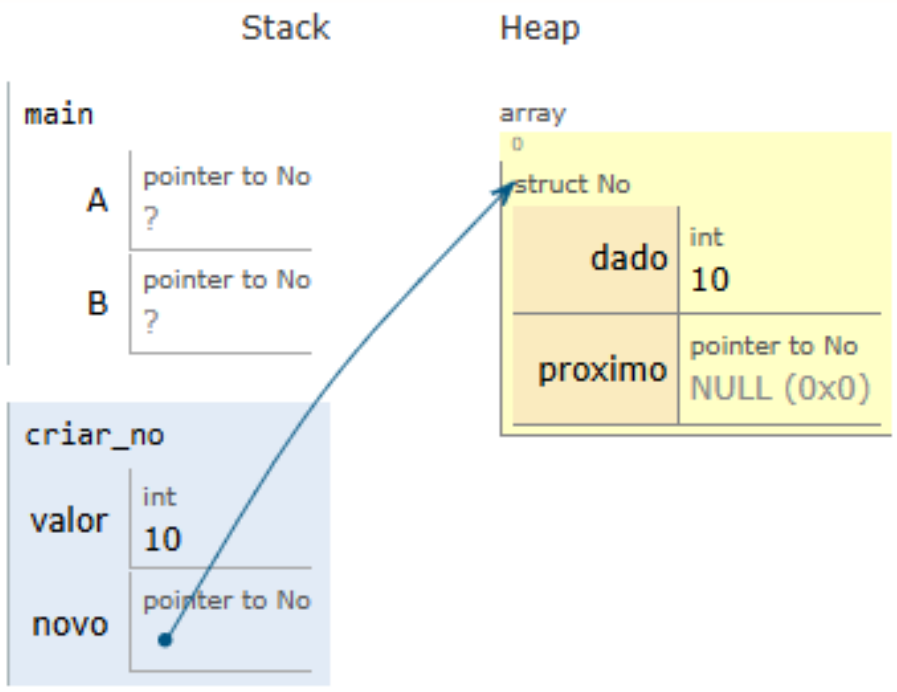
Note: ? refers to an uninitialized value

Note: Pointers to structs, unions, and C++ objects may be ambiguous since a struct/object's address is the same as its first member and a union's address is the same as all its members. Select 'show memory addresses' below to see more details:

C/C++ details: none [default view]

C (C17 + GNU extensions)
known limitations

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct No {
    int dado;
    struct No *proximo;
} No;

No* criar_no(int valor) {

    No *novo = (No*) malloc(sizeof(No));
    if (novo != NULL) {
        novo->dado = valor;
        novo->proximo = NULL;
    }

    return novo;
}

void erro_memoria() {
```



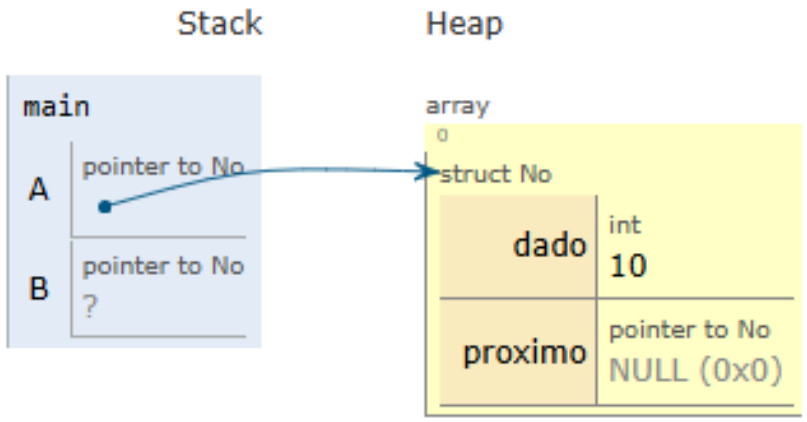
Note: ? refers to an uninitialized value

Note: Pointers to structs, unions, and C++ objects may be ambiguous since a struct/object's address is the same as its first member and a union's address is the same as all its members. Select 'show memory addresses' below to see more details:

C/C++ details: none [default view]

C (C17 + GNU extensions)
known limitations

```
14          novo->proximo = NULL;
15      }
16
17      return novo;
18  }
19
20  void erro_memoria() {
21
22      No *p = criar_no(10);
23
24  }
25
26  int main()
27  {
28      No *A = criar_no(10);
29      No *B = criar_no(20);
30      A->proximo = B;
31      free(B);
32
33      return 0;
34  }
```



Note: ? refers to an uninitialized value

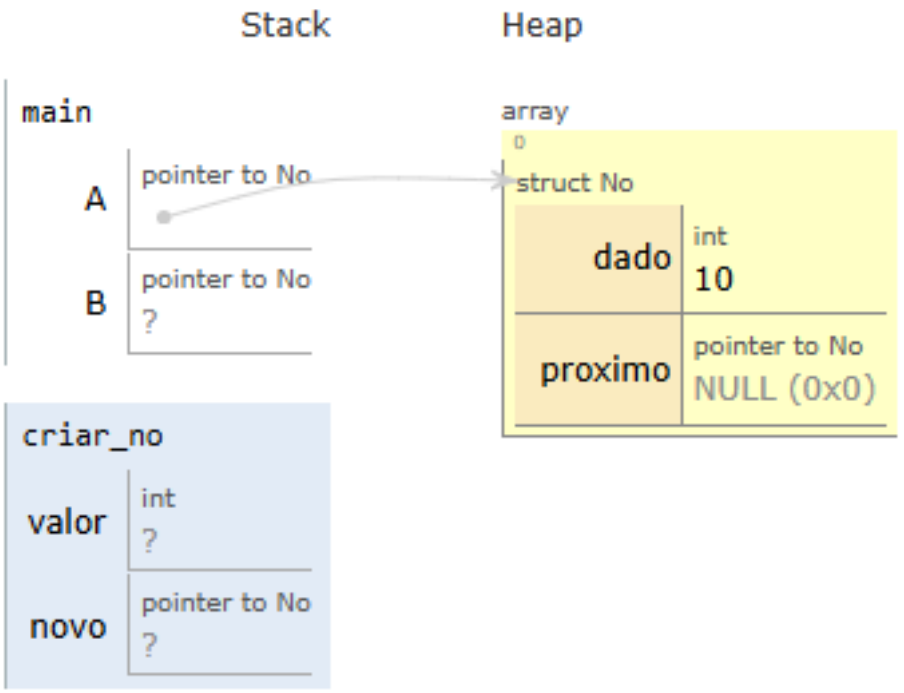
Note: Pointers to structs, unions, and C++ objects may be ambiguous since a struct/object's address is the same as its first member and a union's address is the same as all its members. Select 'show memory addresses' below to see more details:

C/C++ details: none [default view]

C (C17 + GNU extensions)
known limitations

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct No {
    int dado;
    struct No *proximo;
} No;

No* criar_no(int valor) {

    No *novo = (No*) malloc(sizeof(No));
    if (novo != NULL) {
        novo->dado = valor;
        novo->proximo = NULL;
    }

    return novo;
}

void erro_memoria() {
```

Edit this code



Note: ? refers to an uninitialized value

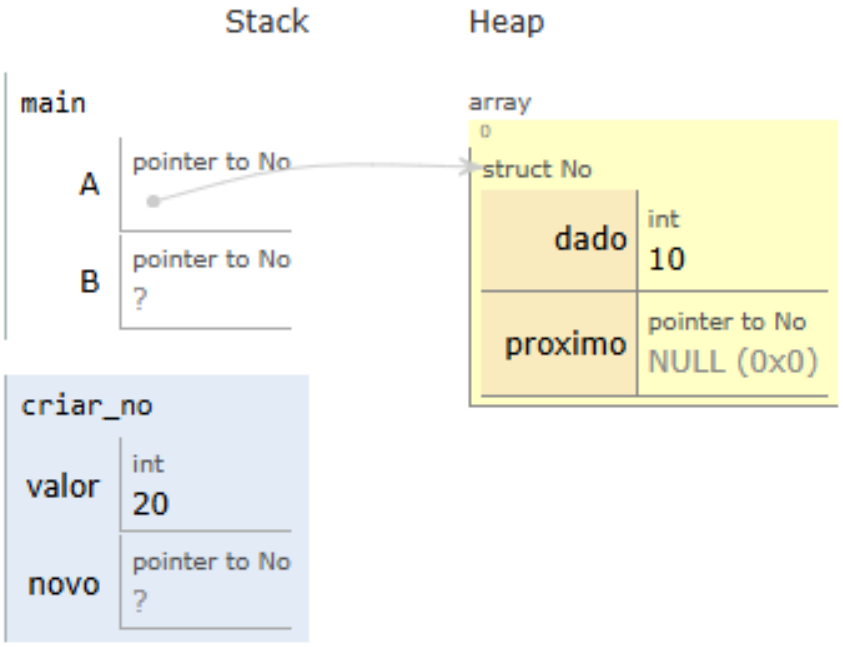
Note: Pointers to structs, unions, and C++ objects may be ambiguous since a struct/object's address is the same as its first member and a union's address is the same as all its members. Select 'show memory addresses' below to see more details:

C/C++ details: none [default view]

C (C17 + GNU extensions)
known limitations

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct No {
    int dado;
    struct No *proximo;
} No;

No* criar_no(int valor) {

    No *novo = (No*) malloc(sizeof(No));
    if (novo != NULL) {
        novo->dado = valor;
        novo->proximo = NULL;
    }

    return novo;
}

void erro_memoria() {
```

Edit this code



Note: ? refers to an uninitialized value

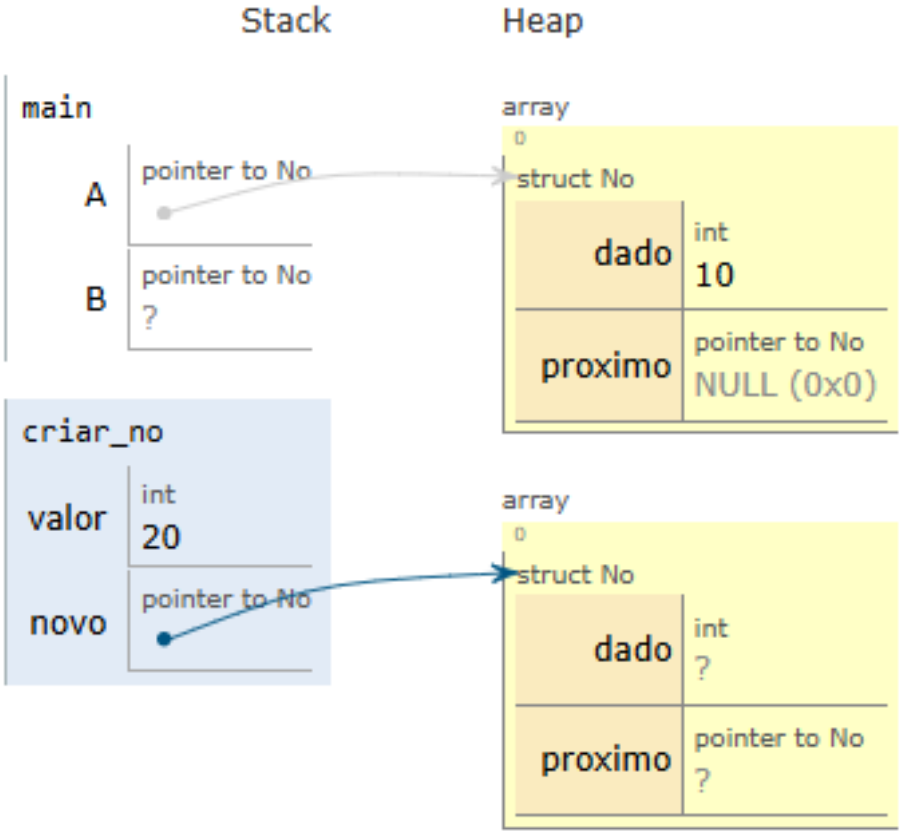
Note: Pointers to structs, unions, and C++ objects may be ambiguous since a struct/object's address is the same as its first member and a union's address is the same as all its members. Select 'show memory addresses' below to see more details:

C/C++ details: none [default view]

C (C17 + GNU extensions)
known limitations

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct No {
    int dado;
    struct No *proximo;
} No;

No* criar_no(int valor) {

    No *novo = (No*) malloc(sizeof(No));
    if (novo != NULL) {
        novo->dado = valor;
        novo->proximo = NULL;
    }

    return novo;
}

void erro_memoria() {
```

Edit this code



Note: ? refers to an uninitialized value

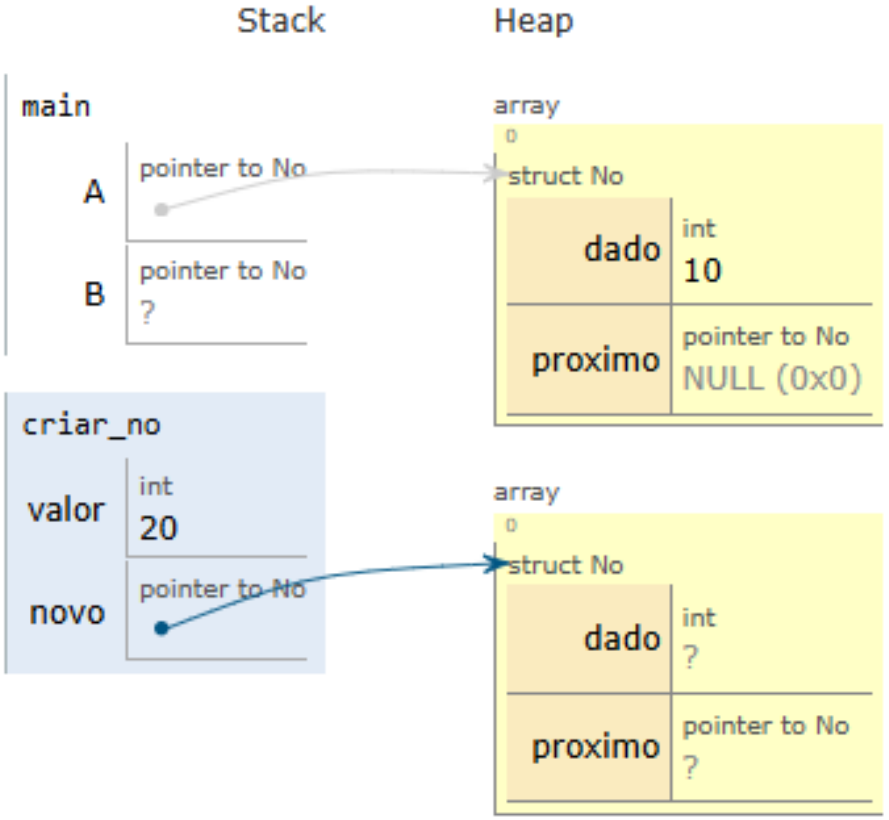
Note: Pointers to structs, unions, and C++ objects may be ambiguous since a struct/object's address is the same as its first member and a union's address is the same as all its members. Select 'show memory addresses' below to see more details:

C/C++ details: none [default view]

C (C17 + GNU extensions)
known limitations

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct No {
    int dado;
    struct No *proximo;
} No;

No* criar_no(int valor) {

    No *novo = (No*) malloc(sizeof(No));
    if (novo != NULL) {
        novo->dado = valor;
        novo->proximo = NULL;
    }

    return novo;
}

void erro_memoria() {
```

Edit this code



Note: ? refers to an uninitialized value

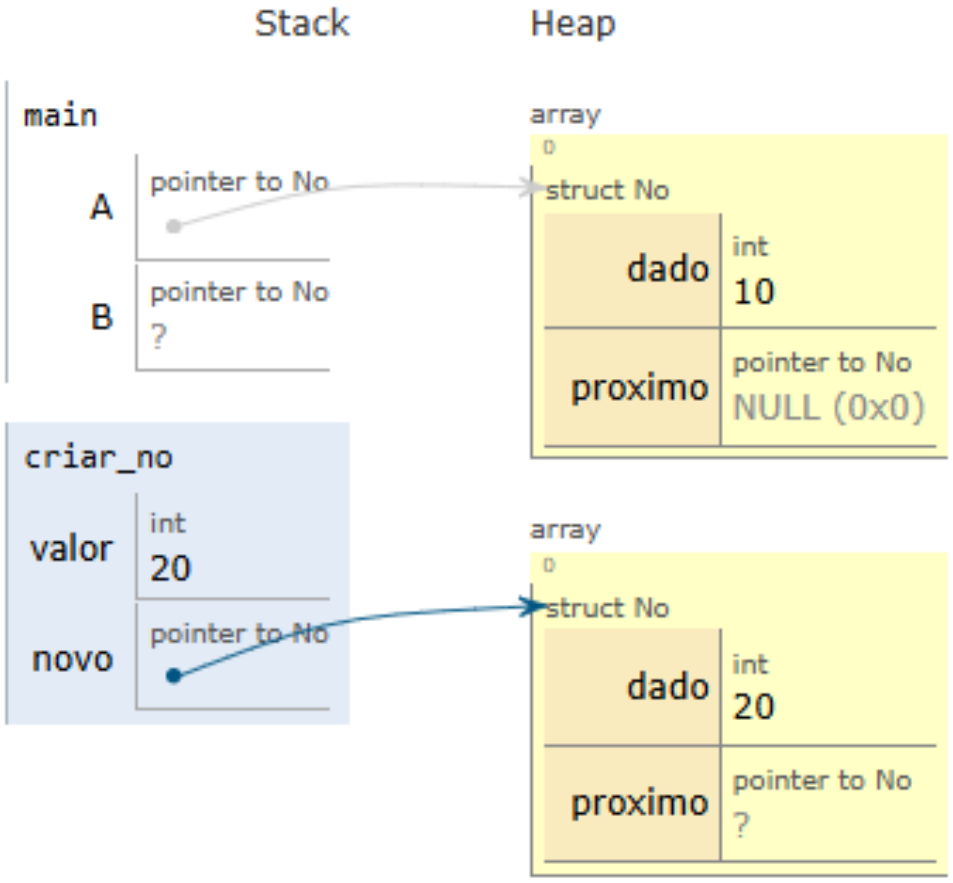
Note: Pointers to structs, unions, and C++ objects may be ambiguous since a struct/object's address is the same as its first member and a union's address is the same as all its members. Select 'show memory addresses' below to see more details:

C/C++ details: none [default view]

C (C17 + GNU extensions)
known limitations

```c
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  typedef struct No {
5      int dado;
6      struct No *proximo;
7  } No;
8
9  No* criar_no(int valor) {
10
11     No *novo = (No*) malloc(sizeof(No));
12     if (novo != NULL) {
13         novo->dado = valor;
14         novo->proximo = NULL;
15     }
16
17     return novo;
18 }
19
20 void erro_memoria() {
```

Edit this code



Note: ? refers to an uninitialized value

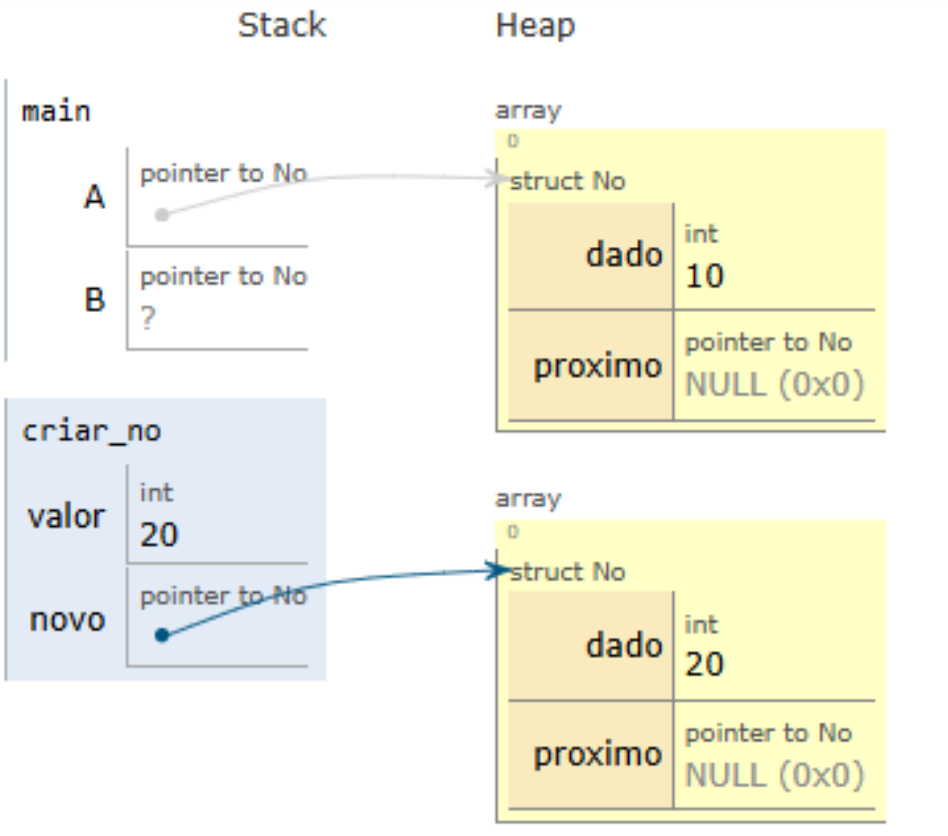
Note: Pointers to structs, unions, and C++ objects may be ambiguous since a struct/object's address is the same as its first member and a union's address is the same as all its members. Select 'show memory addresses' below to see more details:

C/C++ details: none [default view]

C (C17 + GNU extensions)
known limitations

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct No {
    int dado;
    struct No *proximo;
} No;

No* criar_no(int valor) {

    No *novo = (No*) malloc(sizeof(No));
    if (novo != NULL) {
        novo->dado = valor;
        novo->proximo = NULL;
    }

    return novo;
}

void erro_memoria() {
```

Edit this code



Note: ? refers to an uninitialized value

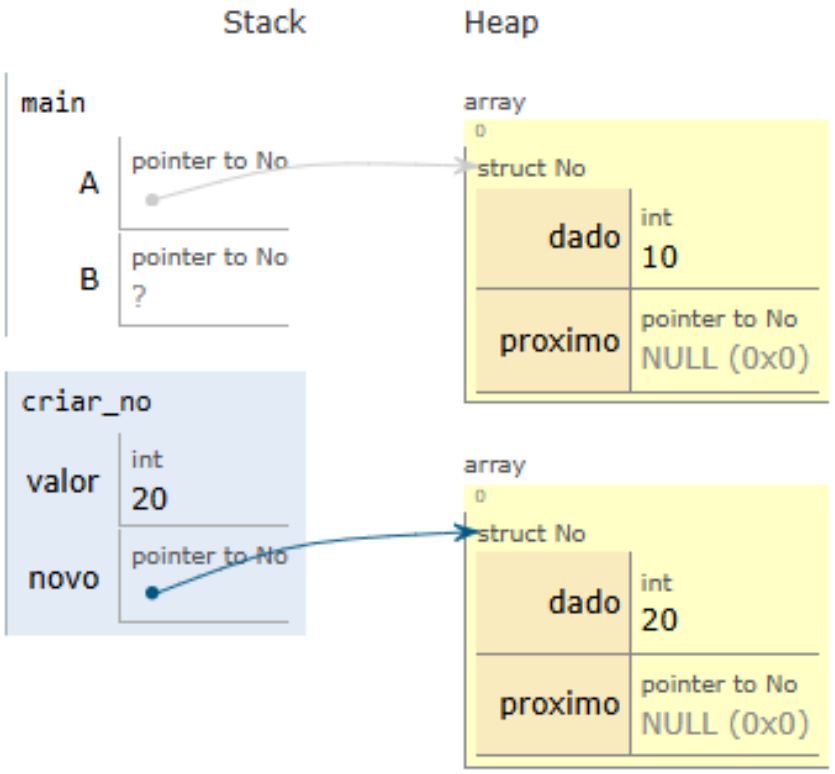
Note: Pointers to structs, unions, and C++ objects may be ambiguous since a struct/object's address is the same as its first member and a union's address is the same as all its members. Select 'show memory addresses' below to see more details:

C/C++ details: none [default view]

C (C17 + GNU extensions)
known limitations

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct No {
    int dado;
    struct No *proximo;
} No;

No* criar_no(int valor) {

    No *novo = (No*) malloc(sizeof(No));
    if (novo != NULL) {
        novo->dado = valor;
        novo->proximo = NULL;
    }

    return novo;
}

void erro_memoria() {
```

Edit this code



Note: ? refers to an uninitialized value

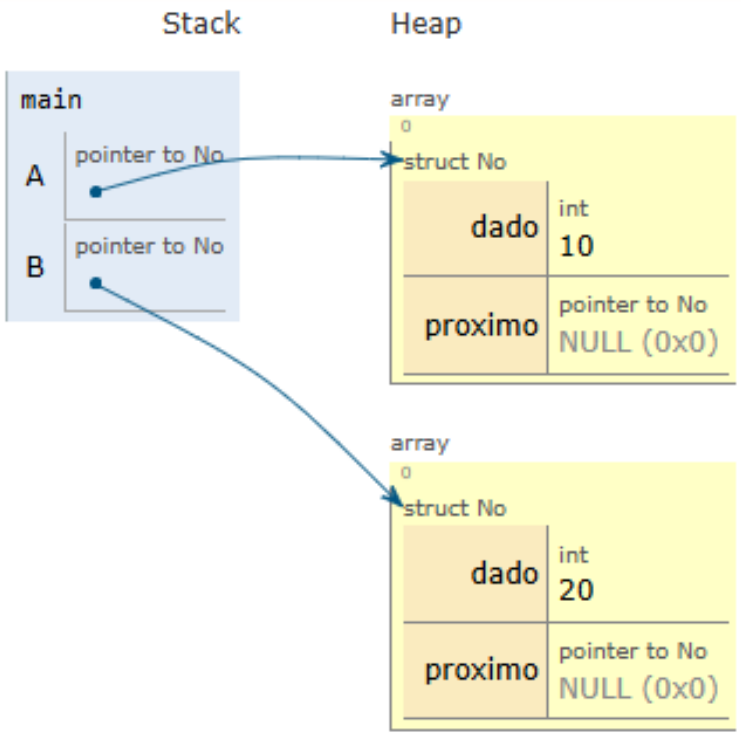
Note: Pointers to structs, unions, and C++ objects may be ambiguous since a struct/object's address is the same as its first member and a union's address is the same as all its members. Select 'show memory addresses' below to see more details:

C/C++ details: none [default view]

C (C17 + GNU extensions)
known limitations

```
14          novo->proximo = NULL;
15      }
16
17      return novo;
18  }
19
20  void erro_memoria() {
21
22      No *p = criar_no(10);
23
24  }
25
26  int main()
27  {
28      No *A = criar_no(10);
29      No *B = criar_no(20);
30      A->proximo = B;
31      free(B);
32
33      return 0;
34  }
```

Edit this code



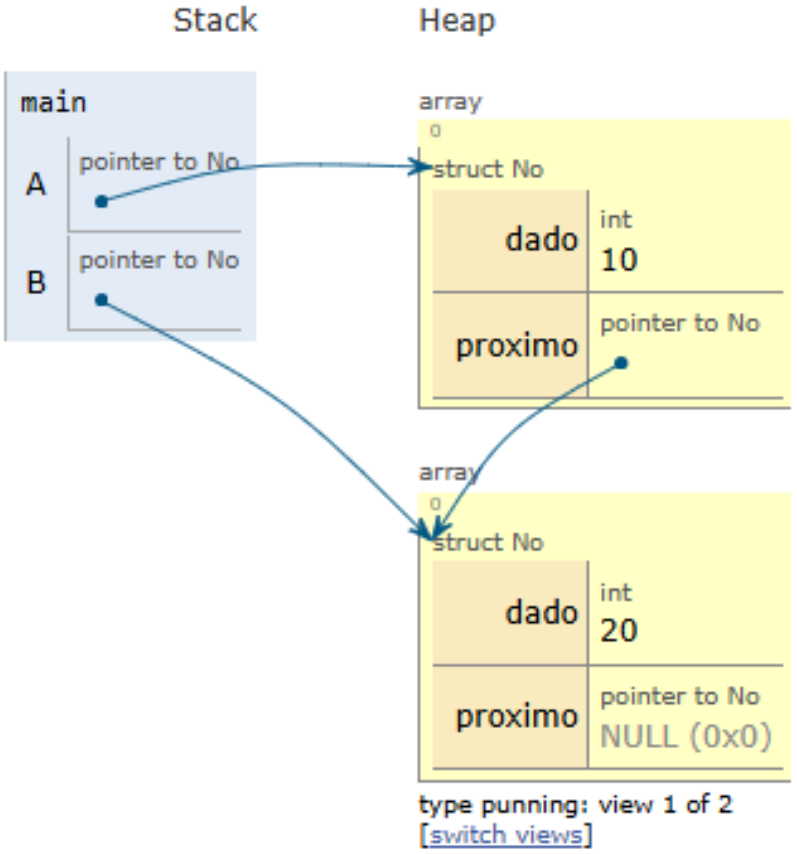
Note: Pointers to structs, unions, and C++ objects may be ambiguous since a struct/object's address is the same as its first member and a union's address is the same as all its members. Select 'show memory addresses' below to see more details:

C/C++ details: none [default view]

C (C17 + GNU extensions)
known limitations

```
14          novo->proximo = NULL;
15      }
16
17      return novo;
18  }
19
20  void erro_memoria() {
21
22      No *p = criar_no(10);
23
24  }
25
26  int main()
27  {
28      No *A = criar_no(10);
29      No *B = criar_no(20);
30      A->proximo = B;
31      free(B);
32
33      return 0;
34  }
```

Edit this code



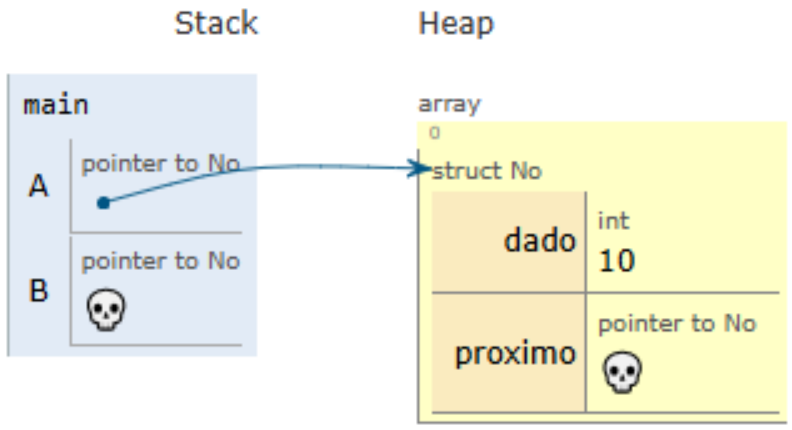
type punning: view 1 of 2
[switch views]

Note: Pointers to structs, unions, and C++ objects may be ambiguous since a struct/object's address is the same as its first member and a union's address is the same as all its members. Select 'show memory addresses' below to see more details:

C/C++ details: none [default view]

C (C17 + GNU extensions)
known limitations

```
14          novo->proximo = NULL;
15      }
16
17      return novo;
18  }
19
20  void erro_memoria() {
21
22      No *p = criar_no(10);
23
24  }
25
26  int main()
27  {
28      No *A = criar_no(10);
29      No *B = criar_no(20);
30      A->proximo = B;
31      free(B);
32
33      return 0;
34  }
```



Note: Pointers to structs, unions, and C++ objects may be ambiguous since a struct/object's address is the same as its first member and a union's address is the same as all its members. Select 'show memory addresses' below to see more details:

Note: 💀 means a pointer points to memory that is either unallocated or misaligned with data boundaries. 💀 locations are *approximate* and may not match the pointer's real address. Select 'byte-level view of data' below to see more details:

C/C++ details: none [default view]

Como a memória de B foi liberada, o ponteiro A, que estava apontado para B, acaba perdendo a referência, podendo causar problema na execução do código.